vasp_diagnose.py

Diagnose VASP-Flow runs and resubmit them with fixes

Part of VASP Workflow · SLURM HPC

Keivan Esfarjani

October 2026 | https://github.com/KeivanS/VASP-Flow

Contents

1 What it does 2

2 Installation 2

3 Input: the folders file 2
- 3.1 Layouts . . . 3

4 Running the diagnosis 3

5 How a step is judged 5
- 5.1 When is a step OK? . . . 5
- 5.2 Why did it fail? . . . 5

6 Failure classes and their fixes 6

7 The fix plan 6
- 7.1 Which steps are redone . . . 6
- 7.2 What the fix does to the folder . . . 7
- 7.3 LOBSTER binary only . . . 7
- 7.4 The job chain . . . 7

8 Output 7
- 8.1 diagnosis.tsv . . . 8
- 8.2 A fix script . . . 8
- 8.3 submit_all_fixes.sh . . . 9

9 Submitting the fixes 10
- 9.1 Undoing a fix . . . 10

10 Typical session 11

11 Limitations and notes 11

1 What it does

`vasp_diagnose.py` looks at a list of VASP-Flow calculation folders, works out which steps finished, which failed and *why*, and writes shell scripts that repair the inputs and resubmit exactly the steps that have to be redone. It does in one pass what is otherwise done by hand after a large high-throughput (HT) campaign: reading OUTCARs and SLURM logs, recognising out-of-memory crashes, time limits, unconverged SCFs, diverged band calculations, broken CHGCARs or failed LOBSTER runs, editing INCARs, moving old outputs aside and rebuilding the job chain.

The tool works in two separate phases:

1. **Diagnosis** (`python3 vasp_diagnose.py folders.txt`). Read-only: nothing in the calculation folders is changed and nothing is submitted. It writes a report (`diagnosis.tsv`), one fix script per folder that needs one, and a master script `submit_all_fixes.sh`.
2. **Submission** (`bash submit_all_fixes.sh`). Only when you run it are INCARs edited, old outputs moved to `stale_<date>/` and jobs submitted.

Between the two you can read every script and decide what to submit.

> **Note.** `vasp_diagnose.py` runs on the cluster (login node), where the folders, `sacct` and `squeue` are. It needs Python $\geq$ 3.6 and only the standard library — no pymatgen, no conda environment.

2 Installation

The script is a single file in the root of the repository. On the cluster:

```
cd ~/VASP-Flow && git pull          # brings vasp_diagnose.py
# optional: keep a copy next to the runs
cp vasp_diagnose.py /scratch/$USER/VASP-Flow/
```

It can be run from any directory; all paths it writes into the scripts are absolute.

3 Input: the folders file

The only required argument is a text file with one folder per line. Empty lines and everything after `##` are ignored; `~` and relative paths are allowed (relative to the directory you run the script from).

```
# folders.txt
/scratch/ke4c/VASP-Flow/HT_1-500        # an HT run: every
    materials/<id> is checked
/scratch/ke4c/VASP-Flow/HT_ins_1
HT_redo_plainRe                         # relative to the current
    directory
~/projects/GaAs_test                    # one single project folder
~/projects                              # a folder of project folders
```

Each line is expanded as follows:

Line points to	What is checked
a **material/project folder**	that folder. A folder counts as one when it contains at least one of `01_relax 02_scf 03_bands 04_dos 06_dfpt 08_lobster`.
an **HT root** (has `materials/`)	every `materials/<id>` that is a material folder.
any **other folder**	each direct sub-folder that is a material folder (e.g. the output of `ht-mp-scf.py`, or a directory of projects).

Lines that are not a folder, or that contain no step folders, are reported with `??` and skipped.

3.1 Layouts

For every material folder the script decides how its jobs are run, because the fix scripts have to resubmit them the same way:

Layout	Recognised by	Resubmitted with
`ht`	`job.sbatch` in the folder (`ht-semimetals.py`)	`sbatch job.sbatch <step>`; walltimes from `TIME_*` in the HT root's `env.sh`; only the steps relax, scf, bands, lobster exist
`slurm`	a step `run.sh` with `##SBATCH` lines (`vasp-agent-slurm.py`)	`sbatch <step>/run.sh`; walltime from its `##SBATCH -time`
`local`	anything else (`vasp-agent.py`)	`(cd <step> && bash run.sh)`, in the foreground, one after the other

Each folder gets a *label* used for its fix script and in the report: `<HT root>__<id>` for `ht`, `<parent>__<folder>` otherwise (e.g. `HT_1-500__mp-2377`, `projects__GaAs_test`).

4 Running the diagnosis

```
cd /scratch/$USER/VASP-Flow
python3 vasp_diagnose.py folders.txt
python3 vasp_diagnose.py folders.txt --since 2026-10-01 --out
    diag_oct
```

Option	Default	Meaning
`--out DIR`	`vasp_diagnose_<date>_<time>`	where the report and the scripts are written
`--since DATE`	14 days ago	start of the `sacct` window (YYYY-MM-DD). Must reach back to the jobs you want diagnosed, otherwise timeouts/OOMs are recognised from the logs only
`--kpar-max N`	8	largest KPAR written by the out-of-memory fix
`--spill-max X`	5	LOBSTER absolute charge spilling (%) above which a material is flagged `spilling_high`
`--retry-unknown`	off	also submit folders whose failure cause was not recognised
`--max-queued N`	200	default throttle of `submit_all_fixes.sh` (jobs in the queue)

Tip. Run it when the queue of the folders you are interested in is empty. Folders that still have jobs queued or running are not diagnosed at all (they are reported as `queued`), so a diagnosis in the middle of a campaign only sees part of the picture. It is harmless to run it as often as you like.

The console output is a short summary:

```
Folders checked: 750   (layouts: ht 750)
  ok               612
  spilling_high     58
  oom               29
  scf_nelm           5
  queued            23
  lobster_fail       9
  timeout            6
  nscf_diverged      4
  unknown            4
Fix scripts: 53 to submit, 4 to review (unknown cause)
  review: .../vasp_diagnose_20261012_1530/fix/HT_ins_1__mp-602.sh
  ...
Report :
   /scratch/ke4c/VASP-Flow/vasp_diagnose_20261012_1530/diagnosis.tsv
Submit : nohup bash .../submit_all_fixes.sh >
   .../submit_all_fixes.log 2>&1 &
```

Each folder is counted once, under the cause of its *first* broken step.

5 How a step is judged

5.1 When is a step OK?

Step	OK when
`01_relax`	OUTCAR contains `reached required accuracy`
`02_scf`	OUTCAR finished (`General timing and accounting`) *and* the SCF converged (`aborting loop because EDIFF is reached`)
`03_bands`	finished, converged, and no OSZICAR energy with $\lvert E\rvert > 10^6$ eV
`04_dos`	as bands, and a non-empty DOSCAR
`06_dfpt`	finished, and OUTCAR has the Born charges or the static dielectric tensor
`08_lobster`	`lobster.out` contains `finished in`. A spilling above `--spill-max` keeps the step OK but is reported

`00_convergence`, `05_wannier` and `07_phonons` are not checked; their presence is noted as `<dir>:unchecked` in the report.

5.2 Why did it fail?

For a step that is not OK, the script reads the newest SLURM logs of that step (`<id>-<id>_<step>-<jobid>.out/.err` for `ht`, `slurm-<jobid>.out/.err` otherwise), `vasp.out`, the tail of OUTCAR and the step's `sacct` record (state, elapsed time, time limit; job name `<folder>_<step>`). The first matching rule decides the class. Up to six error lines are copied into the header of the fix script so you can see what was found.

6 Failure classes and their fixes

Class	Recognised by	Fix written into the script
`quota`	`Disk quota exceeded`, `No space left on device`, `error during write`	resubmitted unchanged — **free space first**
`oom`	`insufficient virtual memory`, `Cannot allocate memory`, OOM kill, UCX `ud_ep.c ... unhandled timeout`, sacct `OUT_OF_MEMORY`	in every redone step: KPAR lowered to the largest divisor of the MPI ranks ≤ min(`--kpar-max`, old KPAR); `--mem=0` (whole node memory)
`mpi_init`	`PMPI_Init`, `MPI_Init_thread` failures (bad node)	resubmitted unchanged
`timeout`	sacct `TIMEOUT`, `DUE TO TIME LIMIT`	walltime ×2 (from the sacct limit); a relax continues from CONTCAR
`wavecar`	WAVECAR does not match (bands, cutoff, lattice)	WAVECAR moved aside; start from CHGCAR
`chgcar`	CHGCAR unreadable / truncated	SCF from scratch (`ISTART=0`, `ICHARG=2`)
`zbrent`	`ZBRENT: fatal error` (relax)	`IBRION` 2 → 1, continue from CONTCAR
`symmetry`	rotation-matrix, POSMAP, SGRCON, PRICELV, RHOSYG errors	`SYMPREC = 1E-4`; `ISYM = 0` if SYMPREC was already set
`edddav`	`Error EDDDAV`, `ZHEGV failed`, non-hermitian sub-space matrix	SCF: `ALGO = All`; relax: `ALGO = Normal`; bands/DOS/LOBSTER: as nscf_diverged
`nscf_diverged`	bands/DOS/LOBSTER NSCF with $\lvert E\rvert > 10^6$ eV	`NBANDS` ≥ 1.3× the SCF's NBANDS (even), `ALGO = Normal`, `NELM` ≥ 150
`relax_unconv`	relax finished without reaching the accuracy	continue from CONTCAR (or the last XDATCAR frame)
`scf_nelm`	SCF hit NELM	`ALGO = All`, `NELM` ≥ 150; all later steps redone
`nscf_unconv`	bands/DOS hit NELM	`ALGO = Normal`, `NELM` ≥ 150
`lobster_fail`	LOBSTER NSCF finished, LOBSTER program did not	only the LOBSTER binary is rerun (see §7.3)
`chgcar_missing`	a later step must be redone but `02_scf/CHGCAR` is missing or truncated	the SCF is redone first
`not_run`	no OUTCAR and no log (e.g. an upstream step failed)	submitted
`unknown`	none of the above	script written but **not** submitted unless `--retry-unknown`; see the error lines in its header
`spilling_high`	LOBSTER finished, spilling > `--spill-max`	reported only, no script

Note. Charge spilling is a property of the LOBSTER basis, not a failed run; rerunning the same inputs gives the same spilling. It is therefore only flagged.

7 The fix plan

7.1 Which steps are redone

- If the **relax** or the **SCF** is broken, that step and *every* later step are redone (the later results depend on them).

- Otherwise only the broken later steps (bands, DOS, DFPT, LOBSTER) are redone.
- Bands, DOS, DFPT and the LOBSTER NSCF start from `02_scf/CHGCAR`. If it is missing or truncated (checked against the FFT grid size), the SCF is redone first and the report shows `scf:ok but 02_scf/CHGCAR missing or incomplete`.
- In the `ht` layout `job.sbatch` only knows relax, scf, bands and lobster; broken DOS/DFPT steps are reported (`job.sbatch cannot run: dos`) but not resubmitted.

7.2 What the fix does to the folder

- **INCAR edits** with a small `set_tag` function: every line setting the tag is removed and `TAG = value` is appended. Only the tags listed in the table above are touched.
- **Old outputs** of every redone step (OUTCAR, OSZICAR, vasprun.xml, CONTCAR, CHGCAR, WAVECAR, DOSCAR, `*.lobster`, ...) are *moved* to `<folder>/stale_<date>/<step>/`. Nothing is deleted; inputs (INCAR, KPOINTS, POSCAR, POTCAR, run scripts) stay where they are.
- **Relax restart**: the newest complete CONTCAR (else the last frame of XDATCAR) becomes `01_relax/POSCAR`; the original is kept once as `POSCAR.orig`.
- **SCF start**: when the relax is not redone but its CHGCAR is incomplete, the relax CHGCAR/WAVECAR are moved aside and the SCF starts from scratch (`ISTART = 0`, `ICHARG = 2`).

7.3 LOBSTER binary only

When the LOBSTER NSCF finished and `08_lobster/WAVECAR` is there, but the LOBSTER program itself failed (`lobster_fail`), the NSCF is not repeated: only `lobster.out`/`*.lobster` are moved aside and `08_lobster/.vf_resume` is set to `post`, which makes the step script skip VASP and run only the LOBSTER binary. (Not available in the `local` layout, which reruns the step.)

7.4 The job chain

Layout	Chain
`ht`	the redone steps in order, each `--dependency=afterok` on the previous one; lobster after bands uses `afterany` (LOBSTER does not need the bands)
`slurm`	scf `afterok` on relax; bands, DOS, DFPT, LOBSTER each `afterok` on the scf (if those are resubmitted)
`local`	run one after the other in the foreground; stops at the first failure

Every job is submitted with `--job-name=<folder>_<step>` and an explicit `--time`. The walltime is the layout's normal one (`env.sh TIME_*` / `##SBATCH -time` / built-in defaults: relax 8 h, scf 4 h, bands/DOS 2 h, DFPT 8 h, LOBSTER 4 h), doubled for a step that timed out.

8 Output

Everything goes into the `--out` directory:

```
vasp_diagnose_20261012_1530/
  diagnosis.tsv            report, one line per folder
  fix/<label>.sh           one fix script per folder that needs one
  submit_all_fixes.sh      submits all the fix scripts
  .submitted/<label>       created by each fix script after a
     successful submission
```

8.1 diagnosis.tsv

Tab-separated, with a header line:

Column	Content
`label`	folder label (also the name of the fix script)
`layout`	`ht`, `slurm` or `local`
`path`	absolute path of the folder
`steps`	`step:status` for every step present, e.g. `relax:ok scf:oom bands:ok lobster:ok`; plus notes (`05_wannier:unchecked`, `job.sbatch cannot run: dos`)
`action`	`ok`, `spilling_high`, `queued`, or `<class> -> <redone steps>`; prefixed with `REVIEW` when the cause is unknown
`script`	path of the fix script (empty when nothing is to be done)

Example lines (path column shortened):

```
HT_1-500__mp-2377  ht  .../mp-2377  relax:ok scf:scf_nelm bands:ok
   lobster:ok  scf_nelm -> scf>bands>lobster
HT_ins_1__mp-4019  ht  .../mp-4019  relax:ok scf:ok bands:ok
   lobster:oom       oom -> lobster
HT_ins_1__mp-602   ht  .../mp-602   relax:unknown scf:ok ...
                     REVIEW unknown -> relax>scf>bands>lobster
HT_1-500__mp-9586  ht  .../mp-9586  relax:ok scf:ok bands:ok
   lobster:ok (spilling 7.1%)  spilling_high
```

Useful one-liners (with `command` in front if your `grep`/`ls` are aliased):

```
awk -F'\t' 'NR>1␣{split($5,a,"␣");␣c[a[1]]++}␣END␣{for␣(k␣in␣c)␣
   print␣k,␣c[k]}' diagnosis.tsv
awk -F'\t' '$5␣~␣/^REVIEW/␣{print␣$1}' diagnosis.tsv          #
   unknown causes
awk -F'\t' '$5␣==␣"spilling_high"␣{print␣$1,␣$4}' diagnosis.tsv
```

8.2 A fix script

Each `fix/<label>.sh` is a self-contained bash script. Its header is a record of the diagnosis; the body (1) checks it is still safe to act, (2) applies the fixes and (3) submits. Example for an SCF that ran out of memory:

```
#!/bin/bash
# vasp_diagnose fix for HT_ins_1__mp-4387
# folder : /scratch/ke4c/VASP-Flow/HT_ins_1/materials/mp-4387
   (layout: ht)
# diagnosed 2026-10-12 15:30:  relax: ok  scf: oom  bands: not_run
   lobster: not_run
#   scf | forrtl: severe (41): insufficient virtual memory
# resubmits: scf -> bands -> lobster
set -u
```

```
D=/scratch/ke4c/VASP-Flow/HT_ins_1/materials/mp-4387; OUT=...;
    LABEL=HT_ins_1__mp-4387
cd "$D" || exit 1
# guard: nothing of this folder queued, outputs unchanged since the
    diagnosis
if squeue ... | grep -q "^mp\-4387_"; then echo "--␣$LABEL:␣jobs␣
    queued,␣skipped"; exit 0; fi
FP=...; if [ "${FP#␣}" != '01_relax/OUTCAR:1791771935␣...' ]; then
    echo "--␣$LABEL:␣outputs␣changed␣since␣the␣diagnosis␣-␣run␣
       vasp_diagnose.py␣again"; exit 0; fi
echo "==␣$LABEL"
... helper functions set_tag, stash, stash_file, continue_relax ...
set_tag 02_scf/INCAR KPAR 8
echo "␣␣scf:␣KPAR␣192␣->␣8␣(192␣ranks)"
stash 02_scf
stash 03_bands
stash 08_lobster
J_scf=$(sbatch --parsable --job-name=mp-4387_scf --time=00:30:00
    --mem=0 job.sbatch scf) || ...
J_bands=$(sbatch --parsable --job-name=mp-4387_bands --time=00:30:00
    --mem=0 \
          --dependency=afterok:$J_scf job.sbatch bands) || ...
J_lobster=$(sbatch --parsable --job-name=mp-4387_lobster
    --time=03:00:00 --mem=0 \
          --dependency=afterany:$J_bands job.sbatch lobster) || ...
mkdir -p "$OUT/.submitted" && touch "$OUT/.submitted/$LABEL"
```

The two **guards** make the scripts safe to run late or twice:

- if any job named `<folder>_*` is in the queue, the script does nothing;
- it records the modification times of every step's OUTCAR (and `lobster.out`) at diagnosis time; if any of them changed since (a job ran or finished in the meantime), it does nothing and asks you to diagnose again.

A fix script can be run on its own: `bash fix/HT_ins_1__mp-4387.sh`.

8.3 submit_all_fixes.sh

The master script calls every fix script in turn:

```
#!/bin/bash
# Runs every fix script written by vasp_diagnose.py; safe to re-run
OUT=/scratch/ke4c/VASP-Flow/vasp_diagnose_20261012_1530
MAX_QUEUED=${MAX_QUEUED:-200}
queued() { squeue -h -u "${USER:-$(whoami)}" 2>/dev/null | wc -l |
    tr -d "␣"; }
run_fix() {
    [ -f "$OUT/.submitted/$(basename␣"$1"␣.sh)" ] && return 0
    while [ "$(queued)" -ge "$MAX_QUEUED" ]; do sleep 60; done
    bash "$1" || echo "!!!␣$(basename␣"$1")␣failed"
}
run_fix .../fix/HT_1-500__mp-2377.sh
run_fix .../fix/HT_ins_1__mp-4387.sh
...
# unknown cause - read the script header first; uncomment to submit:
# run_fix .../fix/HT_ins_1__mp-602.sh
```

- It waits while you have `MAX_QUEUED` or more jobs in the queue (counting *all* your jobs), so it can submit thousands of jobs without hitting the scheduler's per-user limit.
- Fix scripts that already submitted (marker in `.submitted/`) are skipped, so after an interruption simply start it again.
- Scripts with an unknown cause are listed commented out.

9 Submitting the fixes

1. **Read the report.** Check the counts and the `REVIEW` lines in `diagnosis.tsv`.
2. **Read a few fix scripts**, at least one of each class, and every unknown one. The header shows the error lines that were found; the body shows exactly which INCAR tags change and which jobs are submitted. You may edit a script (e.g. a different walltime or KPAR) before submitting.
3. **Decide about the unknown ones**: fix the cause by hand, uncomment their `run_fix` line in `submit_all_fixes.sh` (resubmit as planned), or rerun the diagnosis with `--retry-unknown` to include all of them.
4. **Start the master script under `nohup`**, so it survives logging out (it may wait hours for room in the queue):

```
cd /scratch/$USER/VASP-Flow
nohup bash vasp_diagnose_20261012_1530/submit_all_fixes.sh >
    fixes.log 2>&1 &
MAX_QUEUED=100 nohup bash .../submit_all_fixes.sh > fixes.log
    2>&1 &    # smaller throttle
tail -f fixes.log
```

   For a `local`-layout folder the fix script runs VASP itself; run that one directly on a machine where VASP may run, not on a login node.
5. **Monitor**: `squeue -u $USER`; `fixes.log` shows one `== <label>` block per folder with the changes and job IDs, or why it was skipped (`jobs queued`, `outputs changed`).
6. **When the queue is empty, diagnose again** with a new `--out`. Folders fixed successfully now show `ok`; those that failed again get a new class and a new script. Iterate until only `ok`, `spilling_high` and cases you accept are left.

```
cat fixes.log | awk '/^!!!|^--␣/'          # failures and skipped
    folders
pkill -f submit_all_fixes.sh               # stop submitting
    (submitted jobs keep running)
```

9.1 Undoing a fix

Nothing is deleted. To go back for one folder, cancel its jobs, move the files from `stale_<date>/<step>/` back into `<step>/` and revert the INCAR lines appended at the end of the file (`TAG = value`). `01_relax/POSCAR.orig` is the POSCAR before the first restart.

10 Typical session

```
ssh rivanna
cd ~/VASP-Flow && git pull && cp vasp_diagnose.py
   /scratch/$USER/VASP-Flow/
cd /scratch/$USER/VASP-Flow
printf 'HT_1-500\nHT_ins_1\nHT_ins_W\nHT_redo_plainRe\n' >
   folders.txt
squeue -u $USER | wc -l                          # wait until the runs
   are done
python3 vasp_diagnose.py folders.txt --since 2026-10-01
less vasp_diagnose_*/diagnosis.tsv
less vasp_diagnose_*/fix/HT_ins_1__mp-4387.sh
nohup bash vasp_diagnose_*/submit_all_fixes.sh > fixes.log 2>&1 &
# ... later, when the queue is empty again
python3 vasp_diagnose.py folders.txt --out diag_round2
```

When everything is `ok`, collect and audit the results as usual (`ht_tools/collect_lobster.sh`, `ht_tools/audit_run.sh`).

11 Limitations and notes

- **Job names.** sacct records and queued jobs are matched by the job name `<folder>_<step>` used by the VASP-Flow scripts. Jobs submitted under other names are not seen; the logs are then the only source for the failure cause. Two folders with the same name in different runs share their job names, so a queued job of one also holds back the other.
- **sacct window.** Only jobs since `--since` are known. Without a record a timeout or OOM is still recognised from the log messages, but a timeout's walltime is then doubled from the configured value instead of the actual limit.
- **Not checked**: `00_convergence`, `05_wannier`, `07_phonons`.
- **HT layout**: DOS and DFPT cannot be resubmitted through `job.sbatch`.
- **quota** and **mpi_init** are resubmitted unchanged; make sure the disk has room / the node problem is gone.
- A second failure of the same kind is diagnosed again; the fixes are cumulative (e.g. the walltime is doubled again). An OOM with KPAR already $\leq$ `--kpar-max` is resubmitted with the same KPAR — diagnose again with a smaller `--kpar-max`. A step that keeps failing with `scf_nelm` or `nscf_diverged` after one round needs a look by hand.
- **Charge spilling** is only reported. Lowering it requires a different LOBSTER basis (`basisfunctions` in `lobsterin`), not a rerun.
- The fix scripts use GNU `stat -c` and `sed -i`; they are meant for Linux clusters.